What is function?

In **C++**, a **function** is a self-contained block of code that performs a specific task, can take input (parameters), and may return a value. It improves code modularity, reusability, and readability.

Syntax:

```
return_type function_name(parameters) {
    // Function body
}

Example:
int add(int a, int b) {
    return a + b;
}
```

Types of Functions:

User-defined Function: A **user-defined function** in C++ is a function created by the programmer to perform a specific task. It allows code modularity, reusability, and better organization. The function can take inputs (parameters) and may return a value.

Library Function: A **library function** in C++ is a pre-defined function provided by standard libraries, designed to perform common tasks like input/output, math operations, or string manipulation. These functions save time and reduce code complexity. Example: sqrt(), setw(), strcat(), etc.

Parameter Passing to Functions:

In **C++**, **actual parameters** are the values or variables passed to a function when it is called. These parameters provide the actual data for the function to process.

```
    Multiplication M = new Multiplication();
    int gfg = 5, gfg2 = 10;
    int gfg3 = multiply( gfg, gfg2 );
    cout << "Result is " << gfg3;
  }
}
```

Actual Parameter

In **C++**, **formal parameters** are the variables declared in a function definition that act as placeholders for the values (actual parameters) passed to the function during a function call.

```
class Multiplication {
  int multiply( int x, int y ) { return x * y; }
public
  static void main()
  {
```

Formal Parameter

There are two most popular ways to pass parameters:

In **C++**, **pass by value** is a method of passing arguments to a function where a **copy** of the actual parameter's value is passed. Changes made to the parameter inside the function **do not affect** the original value.

In **C++**, **pass by reference** is a method of passing arguments to a function where the **memory address** of the actual parameter is passed. This allows the function to modify the original value of the argument.

Difference between call by value and call by reference in C++ :

Call by value	Call by reference
A copy of the value is passed to the function	An address of value is passed to the function
Changes made inside the function are not reflected on other functions	Changes made inside the function are reflected outside the function as well
Actual and formal arguments will be created at different memory location	Actual and formal arguments will be created at same memory location.

Example 1: Display a Text

```
#include <iostream>
using namespace std;

void showText() {
    cout << "Hello, this is a text from a function!" << endl;
}

int main() {
    showText();
    return 0;
}
```

Example 2: Function with Parameters

```
#include <iostream>
using namespace std;

void displayNum(int n1, float n2) {
    cout << "The int number is " << n1 << endl;
```

```
    cout << "The double number is " << n2 << endl;
}

int main() {
    int num1 = 5;
    double num2 = 5.5;

    // Calling the function
    displayNum(num1, num2);

    return 0;
}
```

Example 3: Add Two Numbers

```
#include <iostream>
using namespace std;

// Declaring a function
int add(int a, int b) {
    return (a + b);
}

int main() {
    int sum;

    // Calling the function and storing
    // the returned value in sum
    sum = add(100, 78);

    cout << "100 + 78 = " << sum << endl;

    return 0;
}
```

Function prototype : In C++, a function prototype is a declaration of a function that specifies its name, return type, and parameters without providing the function body.

```
#include <iostream>
using namespace std;

// Function prototype
int add(int, int);

int main() {
    int result = add(5, 3);  // Function call
    cout << "The sum is: " << result << endl;
    return 0;
}

// Function definition
```

```
int add(int a, int b) {
    return a + b;
}
```

Benefits of Using User-Defined Functions

- Functions make the code reusable. We can declare them once and use them multiple times.
- Functions make the program easier as each small task is divided into a function.
- Functions increase readability.

C++ User-defined Function Types

For better understanding of arguments and return values in functions, user-defined functions can be categorized into various types:

- Function with no argument and no return value
- Function with no argument but return value
- Function with argument but no return value
- Function with argument and return value

What is Function Overloading in C++?

Two or more functions can have the same name but different parameters; such functions are called function overloading in c++. The function overloading in the c++ feature is used to improve the readability of the code.

Rules of Function Overloading in C++

1.These functions have different parameter type
sum(int a, int b)
sum(double a, double b)

2.These functions have a different number of parameters
sum(int a, int b)
sum(int a, int b, int c)

3.These functions have a different sequence of parameters
sum(int a, double b)
sum(double a, int b)

Why is function overloading in C++ used for?

Function overloading is similar to polymorphism that helps us to get different behavior, with the same name of the function. Function overloading in c++ is used for code reusability and to save memory.

Types of function overloading in c++

There are two types of function overloading in c++-

Compile time overloading– In compile time overloading, functions are overloaded using different signatures. The signature of a function includes its return type, number of parameters and types of parameters.

Runtime overloading -In runtime overloading, functions are overloaded using a different number of parameters.

Causes of function overloading in c++

1) Type Conversion.
2) Function with default arguments.
3) Function with a pass-by reference.

Advantages of Function Overloading in c++

1. The program is very simple and also easier to understand.
2. It saves memory space, maintains consistency, makes clear interface for methods regardless of the parameter's type and readability of the program.
3. Using the function overloading concept, we can develop more than one function with the same name, but the arguments passed should be of different types.
4. Function overloading makes the program faster.
5. Function overloading is used for code reusability and to save memory.

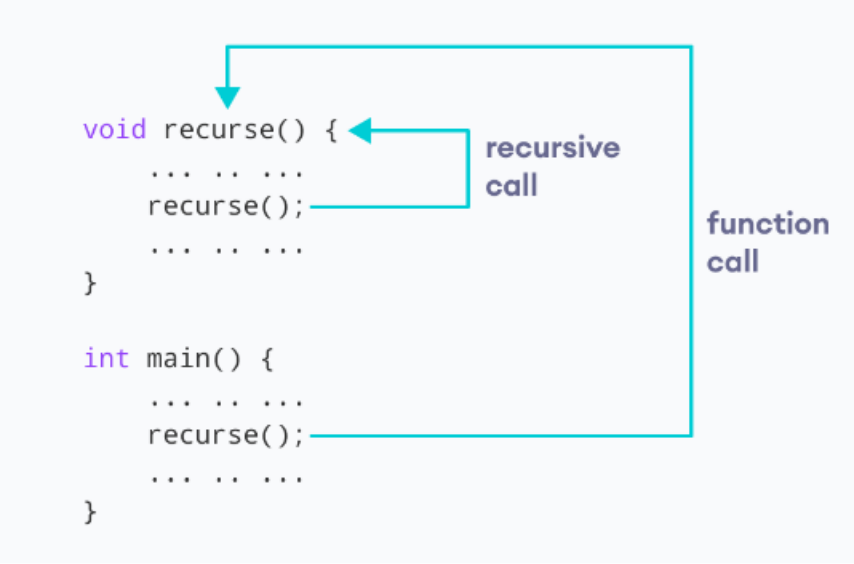
Disadvantages of Function Overloading in c++

1. Function declarations that differ by their return type cannot be overloaded with the function overloading process.
2. If any static member function is declared, then the same parameters or the same name types cannot be overloaded.

C++ Recursion

A function that calls itself is known as a recursive function. And this technique is known as recursion.

The figure below shows how recursion works by calling itself over and over again. The recursion continues until some condition is met.



Advantages of C++ Recursion :

1. It makes our code shorter and cleaner.
2. Recursion is required in problems concerning data structures and advanced algorithms, such as Graph and Tree Traversal.

Disadvantages of C++ Recursion

1. It takes a lot of stack space compared to an iterative program.
2. It uses more processor time.
3. It can be more difficult to debug compared to an equivalent iterative program.

Factorial of a Number Using Recursion

```
#include <iostream>
using namespace std;

// Recursive function to calculate factorial
int factorial(int n) {
    if (n <= 1) return 1;  // Base case
    return n * factorial(n - 1);  // Recursive call
}

int main() {
    int result = factorial(5);
    cout << "Factorial of 5 is: " << result << endl;
    return 0;
}
```

```
return 5 * factorial(4) = 120
└── return 4 * factorial(3) = 24
    └── return 3 * factorial(2) = 6
        └── return 2 * factorial(1) = 2
            └── return 1 * factorial(0) = 1
```

What is pointer ?

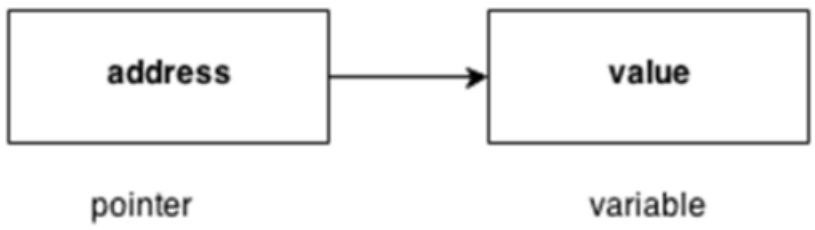
In **C++**, a **pointer** is a variable that stores the memory address of another variable. Rather than holding a data value directly, a pointer holds the location where the value is stored in memory.

```
#include <iostream>
using namespace std;

int main() {
    int var = 10;        // A regular variable
    int *ptr = &var;
```

How to use a pointer?

1. A pointer is declared using the * symbol before the pointer name.
2. To assign a pointer, you use the address-of operator (&) to store the address of a variable.
3. Dereferencing a pointer allows you to access or modify the value stored at the memory address the pointer points to. You do this using the * symbol



Advantage of pointer

- Pointer reduces the code and improves the performance, it is used to retrieve strings, trees etc. and used with arrays, structures, and functions.
- We can return multiple values from a function using pointer.
- It makes you able to access any memory location in the computer's memory.

Usage of pointer

There are many usages of pointers in C++ language.

1. Dynamic memory allocation: In C++ language, we can dynamically allocate memory using malloc() and calloc() functions where pointer is used.

2.Arrays, Functions and Structures: Pointers in C++ language is widely used in arrays, functions and structures. It reduces the code and improves performance.

```
#include <iostream>
using namespace std;

int main() {
    int var = 10;        // A regular variable
    int *ptr = &var;     // Pointer holds the address of var

    cout << "Value of var: " << var << endl;             // Output: 10
    cout << "Address of var: " << &var << endl;           // Address of var
    cout << "Value stored at ptr (dereferencing): " << *ptr << endl;  //
Output: 10

    return 0;
}
```

Pointer Program to swap 2 numbers without using 3rd variable:

```
#include <iostream>
using namespace std;

void swap(int *a, int *b) {
    *a = *a + *b;
    *b = *a - *b;
    *a = *a - *b;
}

int main() {
    int x = 5, y = 10;
    cout << "Before swapping: x = " << x << ", y = " << y << endl;

    swap(&x, &y);  // Swap values using pointers
    cout << "After swapping: x = " << x << ", y = " << y << endl;
    return 0;     }
```

What is a void pointer?
In **C++**, a **void pointer** is a special type of pointer that can point to any data type. It is called "void" because it does not have a specific type associated with it, making it a generic pointer. A void* pointer can store the address of any variable, regardless of its data type.

What is an invalid pointer?
An **invalid pointer** in C++ refers to a pointer that either points to an invalid memory location or has not been initialized properly. Using an invalid pointer can lead to undefined behavior, program crashes, or data corruption. Common types of invalid pointers include:

int *ptr1;
int arr[10];
int *ptr2 = arr+20;
Here, ptr1 is not initialized, making it invalid, and ptr2 is outside of the bounds of arr, making it likewise weak. (Take note that not all build failures are caused by faulty references.)

What is a null pointer?
A **null pointer** in C++ is a pointer that does not point to any valid memory address. Instead, it explicitly points to **no object or data**.

int *ptr1 = 0;
int *ptr2 = NULL;

What is a pointer to a pointer?
In C++, we have the ability to build a pointer to another pointer, which might then point to data or another pointer. The unary operator (*) is all that is needed in the syntax for declaring the pointer for each level of indirection.

char a;
char *b;
char ** c;
a = 'g';
b = &a;
c = &b;
Here b points to a char that stores 'g', and c points to the pointer b.

What is new?
The new is a memory allocation operator, which is used to allocate the memory at the runtime. It returns the starting address of the memory, which gets assigned to the variable. The functionality of the new operator in C++ is similar to the malloc() function, which was used in the C programming language. C++ is compatible with the malloc() function also, but the new operator is mostly used because of its advantages.
Syntax:
```
int* ptr = new int; // Allocates memory for one integer
*ptr = 42;          // Assigns value to allocated memory
std::cout << *ptr; // Outputs: 42
delete ptr;         // Free the allocated memory
```

What is malloc()?
A malloc() is a function that allocates memory at the runtime. This function returns the void pointer, which means that it can be assigned to any pointer type.

The syntax of the malloc() function is given below:
type variable_name = (type *)malloc(sizeof(type));

Differences between the malloc() and new operator:

- The new operator constructs an object, i.e., it calls the constructor to initialize an object while malloc() function does not call the constructor.
- The new is an operator, while malloc() is a predefined function in the stdlib header file.
- The operator new can be overloaded while the malloc() function cannot be overloaded.
- If the sufficient memory is not available in a heap, then the new operator will throw an exception while the malloc() function returns a NULL pointer.
- In the new operator, we need to specify the number of objects to be allocated while in malloc() function, we need to specify the number of bytes to be allocated.
- In the case of a new operator, we have to use the delete operator to deallocate the memory. But in the case of malloc() function, we have to use the free() function to deallocate the memory.

C++ Object:
In C++, an object is a thing created from a class. Think of a class as a blueprint, like a blueprint for a house. The object is the actual house built from that blueprint.

C++ Class:
In C++, a class is like a blueprint for creating objects. It defines the properties and behaviors that the objects created from the class will have. class is a group of similar objects. It is a template from which objects are created.

```
class Student
{
public:
int id; //field or data member
float salary; //field or data member
String name;//field or data member
}
```

Let's see an example of class that has two fields: id and name. It creates instance of the class, initializes the object and prints the object value.

```
#include <iostream>
using namespace std;

class Student {
public:
    int id;
    string name;
};

int main() {
    Student s1;
    s1.id = 201;
    s1.name = "Sonoo Jaiswal";

    cout << s1.id << endl;
    cout << s1.name << endl;
    return 0;   }
```

C++ Constructor:
In C++, a constructor is a special member function of a class that is automatically called when an object of the class is created. Its main purpose is to initialize objects. Constructors have the same name as the class and do not have a return type, not even void.

```
class Car {
public:
    string brand;
    string model;
    int year;

    // Constructor
    Car(string b, string m, int y) {
        brand = b;
        model = m;
        year = y;
    }

    void displayInfo() {
        cout << "Brand: " << brand << endl;
        cout << "Model: " << model << endl;
        cout << "Year: " << year << endl;
    }
};

int main() {
    // Creating an object of Car and calling the constructor
    Car myCar("Toyota", "Corolla", 2021);

    myCar.displayInfo();

    return 0;
}
```

There can be two types of constructors in C++.

- Default constructor
- Parameterized constructor

Default constructor :

A constructor which has no argument is known as the default constructor. It is invoked at the time of creating object.

C++ Parameterized Constructor
A constructor which has parameters is called parameterized constructor. It is used to provide different values to distinct objects.

```
#include <iostream>
using namespace std;

class Employee {
public:
    int id;          // Data member (also called instance variable)
    string name;     // Data member (also called instance variable)
    float salary;    // Data member (also called instance variable)

    // Constructor
    Employee(int i, string n, float s) {
        id = i;
```

```
        name = n;
        salary = s;
    }

    // Member function to display employee details
    void display() {
        cout << id << " " << name << " " << salary << endl;
    }
};

int main(void) {
    // Creating objects of Employee class and initializing them using the
constructor
    Employee e1 = Employee(101, "Sonoo", 890000);
    Employee e2 = Employee(102, "Nakul", 59000);

    // Calling the display function to print the details of each employee
    e1.display();
    e2.display();

    return 0;
}
```

What are the characteristics of a constructor?

- The constructor has the same name as the class it belongs to.
- Although it is possible, constructors are typically declared in the class's public section. However, this is not a must.
- Because constructors don't return values, they lack a return type.
- When we create a class object, the constructor is immediately invoked.
- Overloaded constructors are possible.
- Declaring a constructor virtual is not permitted.
- One cannot inherit a constructor.
- Constructor addresses cannot be referenced to.
- When allocating memory, the constructor makes implicit calls to the new and delete operators.

Structure v/s Class

Structure	Class
If access specifier is not declared explicitly, then by default access specifier will be public.	If access specifier is not declared explicitly, then by default access specifier will be private.

Syntax of Structure:
 	
struct structure_name
{
// body of the structure.
}
	Syntax of Class:
	
	class class_name
	{
	// body of the class.
	}

The instance of the structure is known as "Structure variable".	The instance of the class is known as "Object of the class".

C++ Enumeration
Enum in C++ is a data type that contains fixed set of constants.
It can be used for days of the week (SUNDAY, MONDAY, TUESDAY, WEDNESDAY, THURSDAY, FRIDAY and SATURDAY) , directions (NORTH, SOUTH, EAST and WEST) etc. The C++ enum constants are static and final implicitly.
C++ Enums can be thought of as classes that have fixed set of constants.

C++ Friend function
If a function is defined as a friend function in C++, then the protected and private data of a class can be accessed using the function. For accessing the data, the declaration of a friend function should be done inside the body of a class starting with the keyword friend.
Declaration of friend function in C++

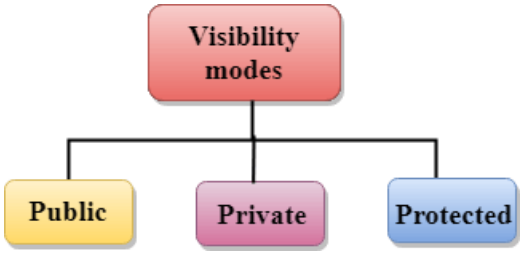
```
class class_name
{
friend data_type function_name(argument/s); // syntax of friend function.
};
```

Characteristics of a Friend function:
- The function is not in the scope of the class to which it has been declared as a friend.
- It cannot be called using the object as it is not in the scope of that class.
- It can be invoked like a normal function without using the object.
- It cannot access the member names directly and has to use an object name and dot membership operator with the member name.
- It can be declared either in the private or the public part.

Inheritance :
In C++, inheritance is a process in which one object acquires all the properties and behaviors of its parent object automatically. In such a way, you can reuse, extend or modify the attributes and behaviors which are defined in other classes.
In C++, the class which inherits the members of another class is called derived class and the class whose members are inherited is called base class. The derived class is the specialized class for the base class.

Here are the different types of inheritance in C++:
1. Single Inheritance: Single inheritance is defined as the inheritance in which a derived class is inherited from only one base class. (A subclass inherits from one superclass.)
2. Multiple Inheritance: Multiple inheritance is the process of deriving a new class that inherits the attributes from two or more classes. (A subclass inherits from more than one superclass.)
3. Multilevel Inheritance: Multilevel inheritance is a process of deriving a class from another derived class. When one class inherits another class which is further inherited by another class, it is known as multi-level inheritance. (A class is derived from another derived class.)
4. Hierarchical Inheritance: (Multiple subclasses inherit from a single superclass.)
5. Hybrid (Virtual) Inheritance: (A combination of two or more types of inheritance to avoid duplication of inherited members.) Hybrid inheritance is a combination of more than one type of inheritance.

Visibility modes can be classified into three categories:



Public: When the member is declared as public, it is accessible to all the functions of the program.
Private: When the member is declared as private, it is accessible within the class only.
Protected: When the member is declared as protected, it is accessible within its own class as well as the class immediately derived from it.

Polymorphism in C++ is like giving a function, object, or variable the ability to take on multiple forms. This concept is essential in object-oriented programming because it allows the same operation to work differently on different classes.

There are two types of polymorphism:
Compile time polymorphism: The overloaded functions are invoked by matching the type and number of arguments. This information is available at the compile time and, therefore, compiler selects the appropriate function at the compile time. It is achieved by function overloading and operator overloading which is also known as static binding or early binding.
Run time polymorphism: Run time polymorphism is achieved when the object's method is invoked at the run time instead of compile time. It is achieved by method overriding which is also known as dynamic binding or late binding.

Compile time vs Run time polymorphism :

Compile time polymorphism	Run time polymorphism
The function to be invoked is known at the compile time.	The function to be invoked is known at the run time.
It is also known as overloading, early binding and static binding.	It is also known as overriding, Dynamic binding and late binding.
Overloading is a compile time polymorphism where more than one method is having the same name but with the different number of parameters or the type of the parameters.	Overriding is a run time polymorphism where more than one method is having the same name, number of parameters and the type of the parameters.
It is achieved by function overloading and operator overloading.	It is achieved by virtual functions and pointers.
It provides fast execution as it is known at the compile time.	It provides slow execution as it is known at the run time.
It is less flexible as mainly all the things execute at the compile time.	It is more flexible as all the things execute at the run time.

Encapsulation in C++
Encapsulation in C++ refers to the wrapping up of data and information in a single unit. In Object Oriented Programming, Encapsulation is defined as binding together the data and the functions that manipulate them.

Two Important property of Encapsulation
Data Protection: Encapsulation protects the internal state of an object by keeping its data members private. Access to and modification of these data members is restricted to the class's public methods, ensuring controlled and secure data manipulation.

Information Hiding: Encapsulation hides the internal implementation details of a class from external code. Only the public interface of the class is accessible, providing abstraction and simplifying the usage of the class while allowing the internal implementation to be modified without impacting external code.

Features of Encapsulation

Below are the features of encapsulation:

1. We cannot access any function from the class directly. We need an object to access that function that is using the member variables of that class.
2. The function which we are making inside the class must use only member variables, only then it is called *encapsulation*.
3. If we don't make a function inside the class which is using the member variable of the class, then we don't call it encapsulation.
4. Encapsulation improves readability, maintainability, and security by grouping data and methods together.
5. It helps to control the modification of our data members.

Data Abstraction in C++

In C++, **data abstraction** is a concept in object-oriented programming that focuses on exposing only the essential features of an object, while hiding the unnecessary details. It helps to reduce complexity and increase efficiency by only providing the necessary information to interact with an object, without revealing the internal implementation.

Data Abstraction is a process of providing only the essential details to the outside world and hiding the internal details, i.e., representing only the essential details in the program.

Data Abstraction can be achieved in two ways:

Abstraction using classes: An abstraction can be achieved using classes. A class is used to group all the data members and member functions into aJ single unit by using the access specifiers. A class has the responsibility to determine which data member is to be visible outside and which is not.

Abstraction in header files: Another type of abstraction is header file. For example, pow() function available is used to calculate the power of a number without actually knowing which algorithm function to calculate the power. Thus, we can say that header files hide all the implementation details from the user.

Advantages Of Abstraction:

1. Implementation details of the class are protected from inadvertent user level errors.
2. A programmer does not need to write low-level code.
3. Data Abstraction avoids code duplication, i.e., programmer does not have to undergo the same tasks every time to perform a similar operation.
4. The main aim of the data abstraction is to reuse the code and the proper partitioning of the code across the classes.
5. Internal implementation can be changed without affecting the user level code.

Access Specifiers Implement Abstraction:

Public specifier: When the members are declared as public, members can be accessed anywhere from the program.

Private specifier: When the members are declared as private, members can only be accessed only by the member functions of the class.

C++ Exception Handling

Exception Handling in C++ is a process to handle runtime errors. We perform exception handling so the normal flow of the application can be maintained even after runtime errors. All exceptions are derived from std::exception class.

Advantage

It maintains the normal flow of the application. In such case, the rest of the code is executed even after exception.

In C++, we use 3 keywords to perform exception handling:

- try
- catch, and
- throw